

PolyAdd.c Explained

🔖 개요

PolyAdd.c는 연결 리스트를 이용해 **다항식드선**을 구현하고, 두 개의 다항식을 변수 호로 드린 후, 같은 차수를 가지는 항은 계수를 바이어 더하는 규별을 가진다.

▶ 구조

```
typedef struct {
    int coef;      // 계수
    int expon;     // 차수
    struct ListNode *link;
} ListNode;

typedef struct {
    int size;
    ListNode *head;
    ListNode *tail;
} ListType;
```

- 항목: 계수 (coef), 차수 (expon)
 - ListType: 다항식 프로경 종합자
-

함수 내용

1. create()

- 다항식 만들기위한 ListType 새로 통을 malloc 통해 만들기

2. insert_last(ListType *plist, int coef, int expon)

- 리스트 마지막에 항 추가
- plist->tail 값을 갱신
- ⚠ 오타 있음: `if (temp = NULL) → if (temp == NULL)`로 수정 필요

3. poly_add(ListType *plist1, ListType *plist2, ListType *plist3)

- 두 리스트 (plist1, plist2)의 head를 a, b로 표기
- while문 내용:

```
while (a && b) {
    if (a->expon == b->expon) {
        sum = a->coef + b->coef;
        if (sum != 0)
```

```
        insert_last(plist3, sum, a->expon);
        a = a->link; b = b->link;
    } else if (a->expon > b->expon) {
        insert_last(plist3, a->coef, a->expon);
        a = a->link;
    } else {
        insert_last(plist3, b->coef, b->expon);
        b = b->link;
    }
}
```

- 마지막: 둘 중 하나를 뺀치 만에 바보 목록을 계속 추가

예제

```
plist1: 3x^5 + 2x^2 + 1
plist2: 4x^5 + 2x^1
result: 7x^5 + 2x^2 + 2x^1 + 1
```

주의 점

- 메모리 할당 시 NULL 확인 오타 (= → ==)
- 다항식은 차수 내림차순으로 구성되어 있어야 함
- 출력 함수는 파일 뒷부분에 별도로 있어야 분석 완료 가능 (생략됨)

개인적 평가

- 그림 구조의 항목 노드가 차수에 따라 정렬되고 더해지는 방식
- 다항식 조사 전 학습에 확장 도움이 되는 구조